

Taller Práctico — Unidad 3

Actividad 1 — Diseño del documento

1. ¿Qué información considera adecuado embeber dentro del documento de un equipo?

Lo más práctico es guardar en un mismo lugar los datos básicos del equipo, como su nombre, tipo, dónde está y en qué estado se encuentra, junto con sus mantenimientos más recientes. Esto porque cuando alguien consulta un equipo, normalmente quiere ver toda esa información de una sola vez, no ir buscándola en partes separadas.

2. ¿Qué información podría separarse en otra colección si el sistema creciera?

Si el sistema creciera exponencialmente, se deberían separar los registros de mantenimientos muy antiguos o el catálogo de repuestos utilizados.

Justificación:

Tiene sentido guardar todo junto cuando la información está relacionada y casi siempre se consulta al mismo tiempo, como los datos del equipo y sus mantenimientos. Pero si con el tiempo ese historial crece demasiado, el archivo se puede volver muy grande y lento de manejar. Por eso, cuando el sistema ya tiene muchos registros acumulados, es mejor guardar ese historial aparte para que todo siga funcionando de forma ágil.

Actividad 2 — Ejemplo de documento JSON

El siguiente JSON muestra cómo se vería registrado un equipo en la base de datos, con su información básica y el historial de mantenimientos incluido dentro del mismo documento. Además de los datos que pide el taller, se agregó un campo extra llamado repuestos usados para listar los materiales que se usaron en cada mantenimiento.

```
{
  "nombre": "Generador Diesel G-02",
  "tipo": "Generador eléctrico industrial",
  "ubicacion": "Planta Sur",
  "estado": "Activo",
  "mantenimientos": [
    {
      "fecha": "2026-01-15",
      "tecnico": "Carlos Martínez",
      "observacion": "Revisión rutinaria y limpieza de inyectores.",
      "repuestos_usados": [
        "Filtro de combustible",
        "Empaques de goma"
      ]
    },
    {
```

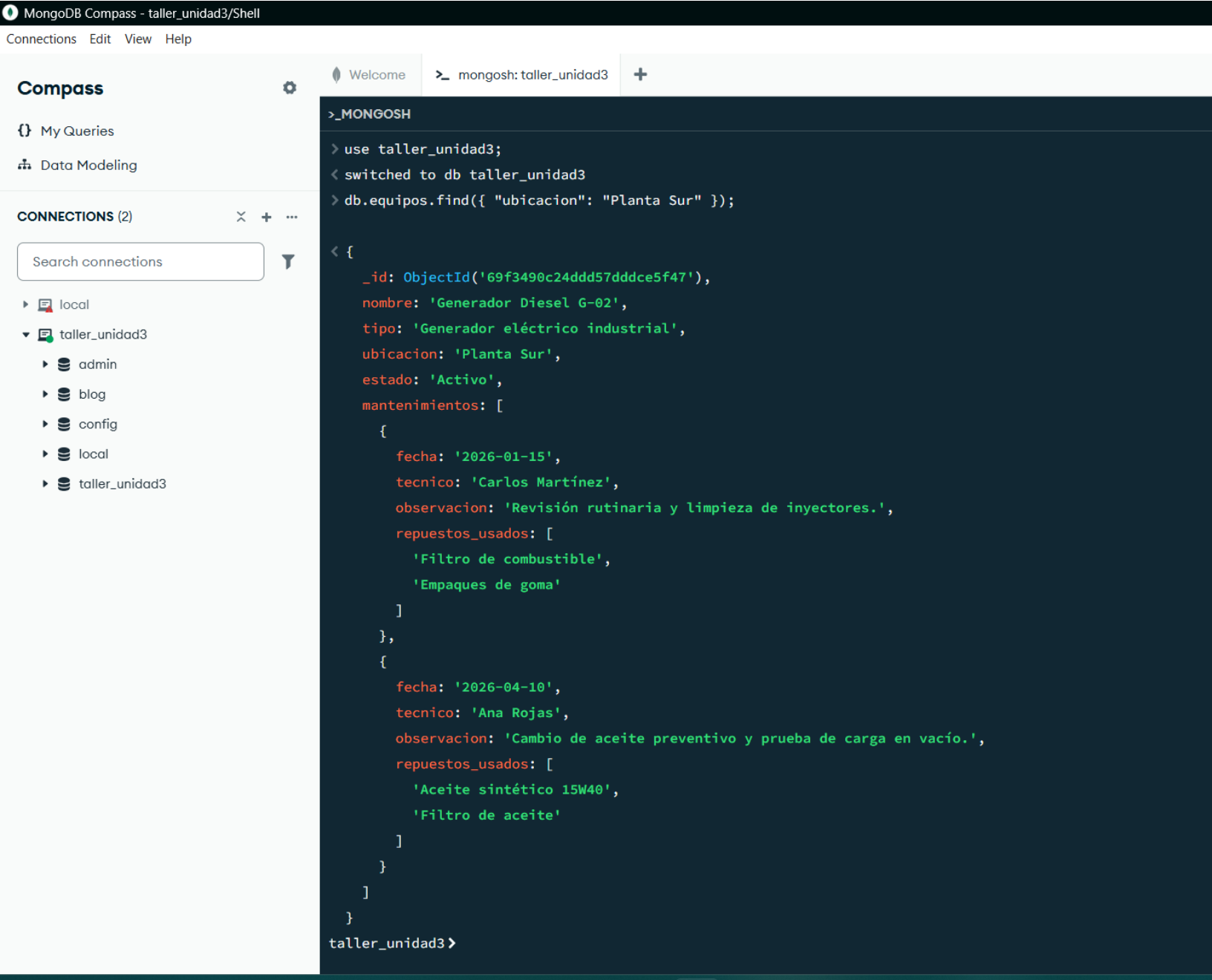
```
"fecha": "2026-04-10",  
"tecnico": "Ana Rojas",  
"observacion": "Cambio de aceite preventivo y prueba de carga en  
vacío.",  
"repuestos_usados": [  
  "Aceite sintético 15W40",  
  "Filtro de aceite"  
]  
}  
]
```

Actividad 3 — Consultas básicas (find)

1. Obtener todos los equipos ubicados en una sede específica

Se filtra directamente por el campo ubicacion usando igualdad exacta. MongoDB recorre la colección y retorna todos los documentos que coincidan.

```
db.equipos.find({ "ubicacion": "Planta Sur" });
```



2. Buscar equipos cuyo nombre contenga una palabra clave (uso de \$regex)

El operador \$regex evalúa una expresión regular sobre el campo indicado. La opción "i" hace la búsqueda insensible a mayúsculas y minúsculas (case-insensitive), por lo que encontrará "generador", "GENERADOR" o cualquier variación.

```
db.equipos.find({ "nombre": { "$regex": "Generador", "$options":  
"i" } }));
```

MongoDB Compass - taller_unidad3/Shell

Connections Edit View Help

Compass

My Queries

Data Modeling

CONNECTIONS (2)

Search connections

- local
- taller_unidad3
 - admin
 - blog
 - config
 - local
 - taller_unidad3

Welcome

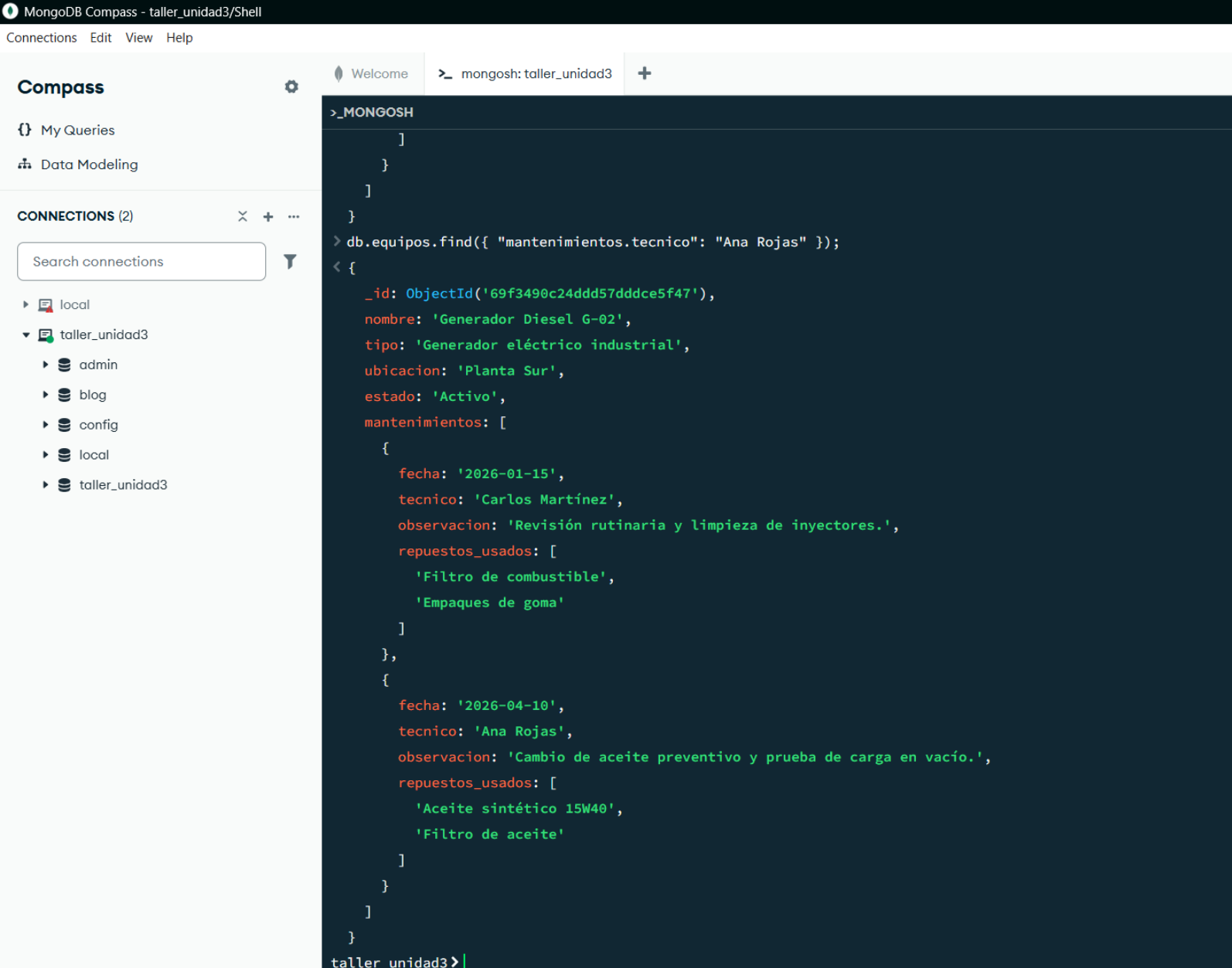
mongosh: taller_unidad3

```
>_MONGOSH  
]  
}  
]  
}  
> db.equipos.find({ "nombre": { "$regex": "Generador", "$options": "i" } });  
< {  
  _id: ObjectId('69f3490c24ddd57dddce5f47'),  
  nombre: 'Generador Diesel G-02',  
  tipo: 'Generador eléctrico industrial',  
  ubicacion: 'Planta Sur',  
  estado: 'Activo',  
  mantenimientos: [  
    {  
      fecha: '2026-01-15',  
      tecnico: 'Carlos Martínez',  
      observacion: 'Revisión rutinaria y limpieza de inyectores.',  
      repuestos_usados: [  
        'Filtro de combustible',  
        'Empaques de goma'  
      ]  
    },  
    {  
      fecha: '2026-04-10',  
      tecnico: 'Ana Rojas',  
      observacion: 'Cambio de aceite preventivo y prueba de carga en vacío.',  
      repuestos_usados: [  
        'Aceite sintético 15W40',  
        'Filtro de aceite'  
      ]  
    }  
  ]  
}  
]  
}  
taller_unidad3>
```

3. Consultar equipos que tengan al menos un mantenimiento realizado por un técnico específico

Se utiliza la notación de punto (dot notation) para acceder al campo tecnico dentro del arreglo mantenimientos. MongoDB itera automáticamente cada elemento del arreglo y retorna el documento completo si encuentra al menos una coincidencia.

```
db.equipo.find({ "mantenimientos.tecnico": "Ana Rojas" });
```



The screenshot shows the MongoDB Compass interface. The left sidebar contains the 'Compass' header, 'My Queries', 'Data Modeling', and a 'CONNECTIONS (2)' section. Under 'CONNECTIONS', there are two connections: 'local' and 'taller_unidad3'. The 'taller_unidad3' connection is expanded, showing a list of databases: 'admin', 'blog', 'config', 'local', and 'taller_unidad3'. The main panel displays the MongoDB Shell with the following code and output:

```
>_MONGOSH
]
}
]
}
> db.equipo.find({ "mantenimientos.tecnico": "Ana Rojas" });
< {
  _id: ObjectId('69f3490c24ddd57dddce5f47'),
  nombre: 'Generador Diesel G-02',
  tipo: 'Generador eléctrico industrial',
  ubicacion: 'Planta Sur',
  estado: 'Activo',
  mantenimientos: [
    {
      fecha: '2026-01-15',
      tecnico: 'Carlos Martínez',
      observacion: 'Revisión rutinaria y limpieza de inyectores.',
      repuestos_usados: [
        'Filtro de combustible',
        'Empaques de goma'
      ]
    },
    {
      fecha: '2026-04-10',
      tecnico: 'Ana Rojas',
      observacion: 'Cambio de aceite preventivo y prueba de carga en vacío.',
      repuestos_usados: [
        'Aceite sintético 15W40',
        'Filtro de aceite'
      ]
    }
  ]
}
```

The output shows two documents from the 'equipo' collection. The first document has a maintenance record by 'Carlos Martínez' on '2026-01-15'. The second document has a maintenance record by 'Ana Rojas' on '2026-04-10'. The shell prompt at the bottom is 'taller_unidad3>|'.

Actividad 4 — Actualización simple (updateOne)

1. Cambiar el estado de un equipo

Se usa `updateOne()` con el operador `$set` para modificar únicamente el campo estado del documento que coincida con el filtro, sin afectar los demás campos.

```
db.equipo.updateOne(  
  { "nombre": "Generador Diesel G-02" },  
  { "$set": { "estado": "En mantenimiento" } }  
);
```



```
taller_unidad3  
└─ equipos  
  
repuestos_usados: [  
  'Aceite sintético 15W40',  
  'Filtro de aceite'  
]  
}  
]  
}  
  
> db.equipo.updateOne(  
  { "nombre": "Generador Diesel G-02" },  
  { "$set": { "estado": "En mantenimiento" } }  
);  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
taller_unidad3>
```

2. Agregar un nuevo registro de mantenimiento al arreglo

El operador \$push agrega un nuevo elemento al final del arreglo mantenimientos sin sobrescribir los registros existentes. Esto es ideal para acumular el historial de forma incremental.

```
db.equipo.updateOne(
  { "nombre": "Generador Diesel G-02" },
  {
    "$push": {
      "mantenimientos": {
        "fecha": "2026-05-01",
        "tecnico": "Luis Fernández",
        "observacion": "Reemplazo de batería principal por desgaste.",
        "repuestos_usados": ["Batería 12V 100Ah"]
      }
    }
  }
)
```

local
taller_unidad3
equipos

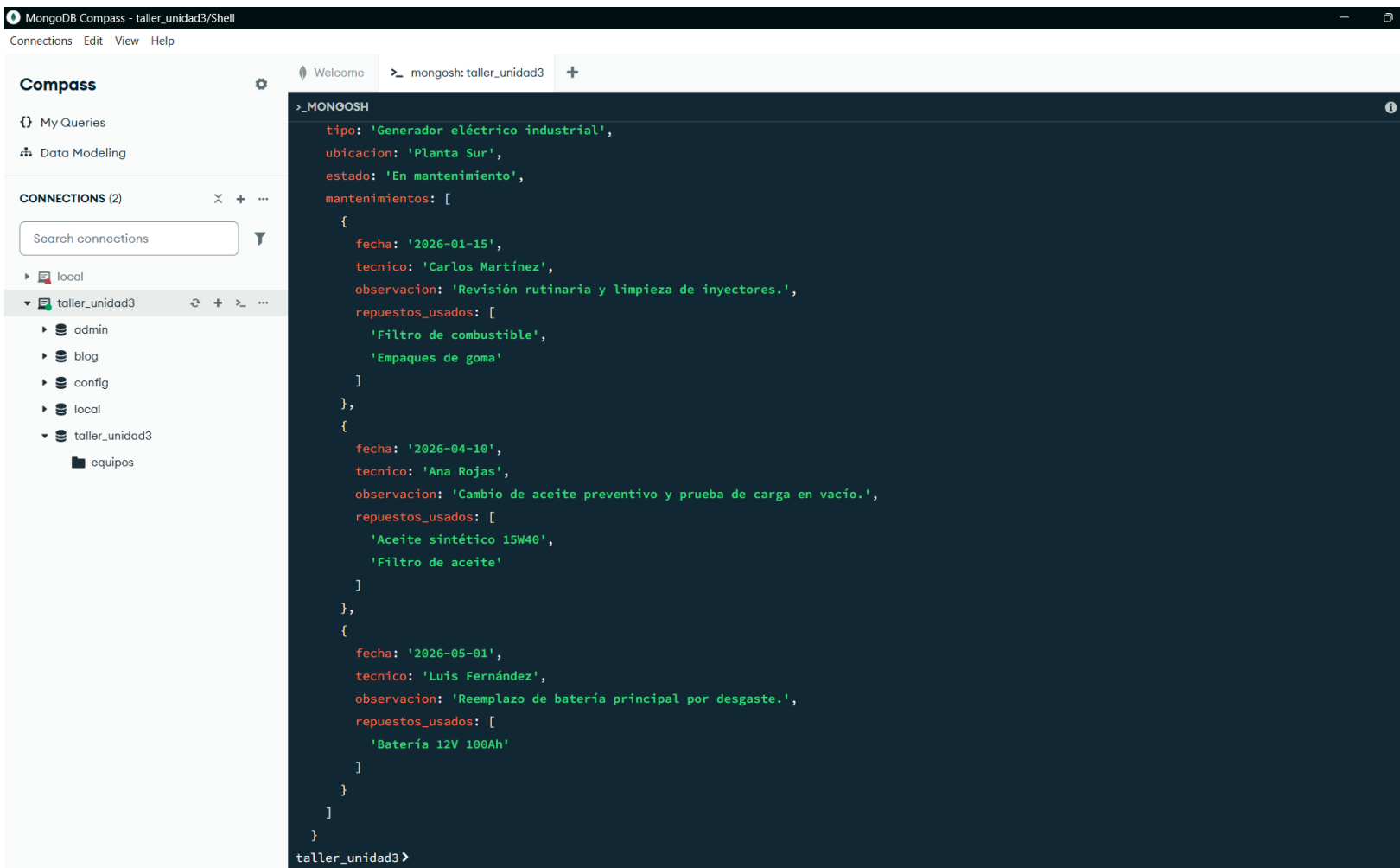
```
> db.equipo.updateOne(
  { "nombre": "Generador Diesel G-02" },
  {
    "$push": {
      "mantenimientos": {
        "fecha": "2026-05-01",
        "tecnico": "Luis Fernández",
        "observacion": "Reemplazo de batería principal por desgaste.",
        "repuestos_usados": ["Batería 12V 100Ah"]
      }
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
taller_unidad3>
```

3. Comprobación de las actualizaciones. Paso a paso:

Se realiza una consulta básica utilizando `find()` filtrando por el nombre del equipo.

Esto devolverá el documento completo, permitiendo confirmar visualmente que el campo estado se modificó correctamente y que el arreglo mantenimientos ahora incluye el nuevo registro.

```
db.equipo.find({ nombre: "Generador Eléctrico G-02" })
```



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS (2)' panel lists 'local' and 'taller_unidad3'. The 'taller_unidad3' connection is expanded, showing a database named 'equipos'. The main panel displays the result of a query in the 'MONGOSH' shell. The query is `db.equipo.find({ nombre: "Generador Eléctrico G-02" })`. The result is a JSON document with the following fields: `tipo` (Generador eléctrico industrial), `ubicacion` (Planta Sur), `estado` (En mantenimiento), and `mantenimientos` (an array of three maintenance records). The first record is dated '2026-01-15' by 'Carlos Martínez'. The second record is dated '2026-04-10' by 'Ana Rojas'. The third record is dated '2026-05-01' by 'Luis Fernández'.

```
>_MONGOSH
{
  tipo: 'Generador eléctrico industrial',
  ubicacion: 'Planta Sur',
  estado: 'En mantenimiento',
  mantenimientos: [
    {
      fecha: '2026-01-15',
      tecnico: 'Carlos Martínez',
      observacion: 'Revisión rutinaria y limpieza de inyectores.',
      repuestos_usados: [
        'Filtro de combustible',
        'Empaques de goma'
      ]
    },
    {
      fecha: '2026-04-10',
      tecnico: 'Ana Rojas',
      observacion: 'Cambio de aceite preventivo y prueba de carga en vacío.',
      repuestos_usados: [
        'Aceite sintético 15W40',
        'Filtro de aceite'
      ]
    },
    {
      fecha: '2026-05-01',
      tecnico: 'Luis Fernández',
      observacion: 'Reemplazo de batería principal por desgaste.',
      repuestos_usados: [
        'Batería 12V 100Ah'
      ]
    }
  ]
}
```

taller_unidad3>

Actividad 5 — Reflexión breve

¿Por qué el modelo de documentos es adecuado para este tipo de información? ¿Qué ventaja ofrece frente a un modelo relacional en este caso?

Este tipo de base de datos funciona muy bien para este caso porque no obliga a que todos los equipos tengan exactamente los mismos campos. Cada uno puede tener la información que le corresponde sin afectar a los demás. Por ejemplo, un mantenimiento puede incluir repuestos y otro no, y eso no genera ningún problema. A diferencia de una base de datos tradicional, donde habría que buscar la información en varias tablas al mismo tiempo para armar un solo resultado, aquí todo está junto en un solo lugar, lo que hace las consultas más simples y rápidas.